

Meta Prompting for AI Systems

Yifan Zhang¹ Yang Yuan Andrew Chi-Chih Yao^{1,2,†}

¹IIS, Tsinghua University ²Shanghai Qi Zhi Institute

Abstract

We introduce Meta Prompting (MP), a framework that emphasizes the formal structure of a task rather than content-specific worked examples. We give a categorical formalization in which a functor maps typed task transformations to typed prompt transformations. Functoriality encodes preservation of identities and composition, but it does not by itself guarantee semantic correctness. We extend MP to Recursive Meta Prompting (RMP), in which a proposer model generates candidate prompt edits, a validator enforces the edit schema, and an executor model uses the resulting prompt. We model the accumulation of valid edit scripts with the Writer monad and their application by a monoid action on prompts. This construction makes accumulated edits independent of parenthesization; it does not guarantee convergence or improved task accuracy. Empirically, a Qwen-72B base model guided by example-free meta-prompts attains 46.3% pass@1 on MATH and 83.5% accuracy on GSM8K. Separately, an MP-CR agent synthesizes a tool-executed Game of 24 solver; the original experiment reports 100% success on 1,362 instances, subject to exact re-verification from the corresponding input and per-instance outputs. The batch amortizes one LLM response across all puzzles, so its token accounting is not protocol-matched to per-instance few-shot or search-based methods.

Project Page: <https://github.com/meta-prompting/meta-prompting>

1 Introduction

The advent of foundation models, particularly large language models (LLMs), has transformed research across computational linguistics, text understanding, and text generation (Devlin et al., 2018; Radford et al., 2018, 2019; Brown et al., 2020; Raffel et al., 2020; OpenAI, 2023). Despite these advances, LLMs remain unreliable on many tasks that require long, precise chains of reasoning, including advanced mathematics (Lightman et al., 2023). This motivates methods that make the intended decomposition and output structure explicit.

A core challenge is that the autoregressive next-token objective does not explicitly require a model to expose, verify, or preserve a multi-step solution plan (Radford et al., 2018, 2019; Brown et al., 2020). The dual-process distinction between rapid, intuitive responses and deliberate, systematic reasoning offers a useful analogy (Kahneman, 2011), although it is not a mechanistic model of LLM computation. Prompting methods can nevertheless use this analogy to motivate explicit scaffolds for decomposition and verification.

Approaches such as Chain-of-Thought (CoT) (Wei et al., 2022) elicit intermediate reasoning steps, while Tree-of-Thought (ToT) (Yao et al., 2023; Long, 2023) organizes search over multiple candidate continuations. These methods can use either demonstrations or zero-shot instructions. They are

```

Integrate step-by-step reasoning to solve mathematical problems under the following structure:
{
  "Problem": "[question to be answered]",
  "Solution": {
    "Step 1": "Begin the response with Let's think step by step.",
    "Step 2": "Follow with the reasoning steps, ensuring the solution process is broken down clearly and logically.",
    "Step 3": "End the solution with the final answer encapsulated in a LaTeX-formatted box,  $\boxed{\dots}$ , for clarity and emphasis."
  },
  "Final Answer": "[final answer to the problem]"
}

```

Figure 1 A structured meta-prompt presented in JSON-like format.

primarily operational prompting and search procedures, however, and do not by themselves specify typed task and prompt categories or a composition-preserving compiler between them.

In response, we introduce *Meta Prompting (MP)*, a paradigm that shifts the focus from content-based analogy to explicit procedural structure. Instead of supplying worked examples, a meta-prompt specifies a reusable schema for decomposing a task and formatting its solution. We formalize a *functorial* meta-prompting scheme from a category of typed task transformations to a category of typed prompt transformations. The functor laws make compositionality an explicit design constraint: composed task transformations must map to composed prompt transformations. This is a structural statement and does not, on its own, guarantee that an LLM will solve the task correctly.

We also apply the framework recursively, a process we call *Recursive Meta Prompting (RMP)*. Analogous to metaprogramming, RMP uses a proposer LLM to generate or refine a prompt. An executor LLM then consumes the result. We model the accumulation of schema-preserving edit scripts with a Writer monad and model the execution of those edits by a monoid action. The construction provides algebraically consistent bookkeeping for recursive edits; it does not imply that refinement converges or monotonically improves task performance.

We evaluate the empirical component of the framework in proof-of-concept experiments on the Game of 24 (Yao et al., 2023), GSM8K (Cobbe et al., 2021), and MATH (Hendrycks et al., 2021).

In summary, our contributions are as follows:

- We introduce Meta Prompting (MP) and formalize a functorial version of MP using category theory. The formalization makes identity preservation and compositionality explicit, while carefully limiting the guarantee to structural equality in the prompt category. We further introduce Recursive Meta Prompting (RMP), model edit accumulation with a Writer monad, and model edit application with a compatible monoid action.
- In proof-of-concept experiments, a Qwen-72B *base* model equipped with example-free meta-prompts obtains 46.3% pass@1 on MATH and 83.5% accuracy on GSM8K without additional instruction tuning. Separately, an MP-CR agent generates an exact Python solver, and the original Game of 24 experiment reports 100% success on 1,362 puzzles. This aggregate requires exact re-verification from the corresponding input and per-instance outputs. The setting is tool-assisted,

batched program synthesis and is not protocol-matched to per-puzzle CoT or ToT evaluation.

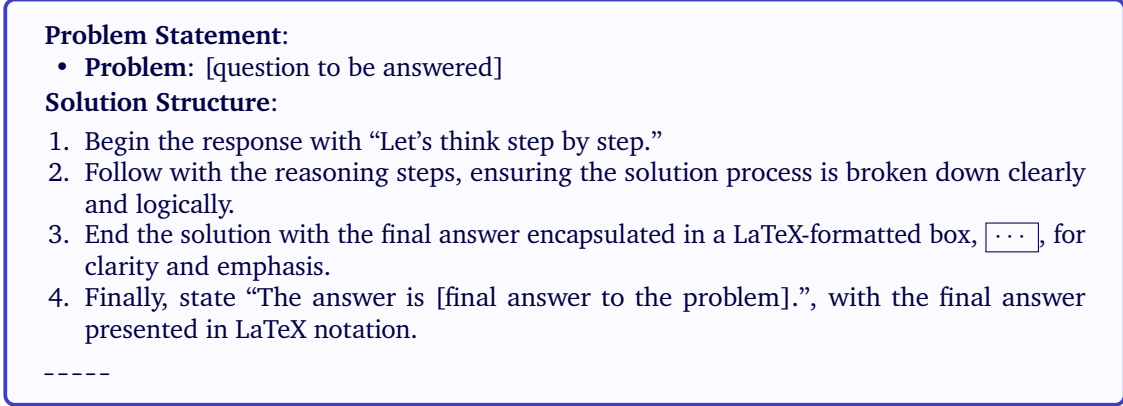


Figure 2 A structured meta-prompt in Markdown format for solving MATH (Hendrycks et al., 2021) problems, following the output style used in the Minerva study (Lewkowycz et al., 2022).

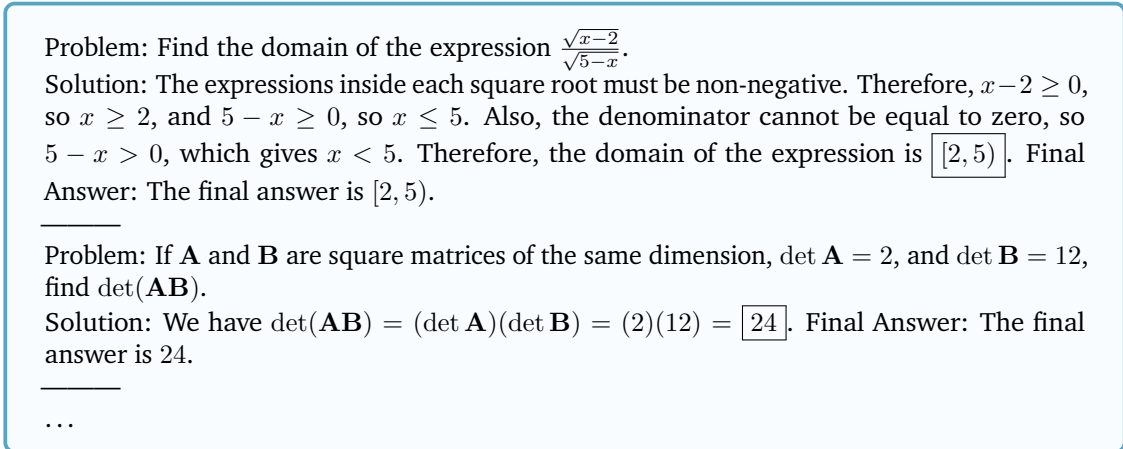


Figure 3 A representative few-shot prompt for solving MATH problems. **Note:** In contrast, our meta-prompt in Fig. 2 is *generated via RMP* from a single task-agnostic meta-meta-prompt (Sec. 4).

2 Background

Category theory provides a high-level language for describing mathematical structures and their relationships. We use it to formalize the relationship between task structures and prompt structures.

2.1 Category Theory

Definition 2.1 (Locally Small Category). A *locally small category* $\mathcal{C}$ consists of a class of objects $\text{Ob}(\mathcal{C})$; for every pair $A, B \in \text{Ob}(\mathcal{C})$, a set of morphisms $\text{Hom}_{\mathcal{C}}(A, B)$; a composition map

$$\text{Hom}_{\mathcal{C}}(B, C) \times \text{Hom}_{\mathcal{C}}(A, B) \longrightarrow \text{Hom}_{\mathcal{C}}(A, C), \quad (g, f) \longmapsto g \circ f;$$

and, for every object A , an identity morphism $\text{id}_A \in \text{Hom}_{\mathcal{C}}(A, A)$. Composition is associative, and the identity morphisms satisfy the left- and right-identity laws.

Definition 2.2 (Morphism). For objects $A, B \in \text{Ob}(\mathcal{C})$, a morphism f from A to B is written $f : A \rightarrow B$. The object A is the domain (or source), and B is the codomain (or target). No disjointness assumption on hom-sets is required; the domain and codomain are part of the typing of a morphism.

Definition 2.3 (Composition of Morphisms). If $f : A \rightarrow B$ and $g : B \rightarrow C$, their composite is $g \circ f : A \rightarrow C$. For composable morphisms $f : A \rightarrow B$, $g : B \rightarrow C$, and $h : C \rightarrow D$, associativity requires

$$h \circ (g \circ f) = (h \circ g) \circ f.$$

Definition 2.4 (Identity Morphisms). For every object A , the identity morphism $\text{id}_A : A \rightarrow A$ satisfies, for every $f : A \rightarrow B$,

$$\text{id}_B \circ f = f = f \circ \text{id}_A.$$

The identity morphism of each object is unique.

2.2 Functors

Definition 2.5 (Covariant Functor). A *covariant functor* $F : \mathcal{A} \rightarrow \mathcal{B}$ assigns

- to each object $A \in \text{Ob}(\mathcal{A})$ an object $F(A) \in \text{Ob}(\mathcal{B})$; and
- to each morphism $m : A_1 \rightarrow A_2$ in $\mathcal{A}$ a morphism $F(m) : F(A_1) \rightarrow F(A_2)$ in $\mathcal{B}$.

The assignments must satisfy

$$F(\text{id}_A) = \text{id}_{F(A)} \quad \text{and} \quad F(m_2 \circ m_1) = F(m_2) \circ F(m_1)$$

for every object A and every composable pair m_1, m_2 .

Definition 2.6 (Contravariant Functor). A *contravariant functor* from $\mathcal{A}$ to $\mathcal{B}$ is equivalently a covariant functor $F : \mathcal{A}^{\text{op}} \rightarrow \mathcal{B}$. Thus, a morphism $m : A_1 \rightarrow A_2$ is mapped to $F(m) : F(A_2) \rightarrow F(A_1)$, identities are preserved, and

$$F(m_2 \circ m_1) = F(m_1) \circ F(m_2).$$

2.3 Natural Transformations

Definition 2.7 (Natural Transformation). Let $F, G : \mathcal{A} \rightarrow \mathcal{B}$ be covariant functors. A *natural transformation* $\alpha : F \Rightarrow G$ is a family of morphisms $\{\alpha_A : F(A) \rightarrow G(A)\}_{A \in \text{Ob}(\mathcal{A})}$ such that, for every morphism $f : A \rightarrow A'$ in $\mathcal{A}$,

$$G(f) \circ \alpha_A = \alpha_{A'} \circ F(f).$$

If every component α_A is an isomorphism, then α is a *natural isomorphism*.

This concept is used below for the unit η and multiplication μ of the RMP Writer monad.

2.4 Monads in Category Theory

Definition 2.8 (Monad). A *monad* on a category $\mathcal{C}$ is a triple (T, η, μ) consisting of

- an endofunctor $T : \mathcal{C} \rightarrow \mathcal{C}$;
- a natural transformation $\eta : \text{Id}_{\mathcal{C}} \Rightarrow T$, called the *unit*; and
- a natural transformation $\mu : T \circ T \Rightarrow T$, called the *multiplication*.

For every object X , these components satisfy the two unit laws and associativity law

$$\begin{aligned}\mu_X \circ T(\eta_X) &= \text{id}_{T(X)}, & \mu_X \circ \eta_{T(X)} &= \text{id}_{T(X)}, \\ \mu_X \circ T(\mu_X) &= \mu_X \circ \mu_{T(X)}.\end{aligned}$$

In computer science, monads provide a compositional interface for structured computations. Below, we use the Writer monad specifically to record prompt-edit histories.

3 Meta Prompting

Meta Prompting is a prompting technique that emphasizes the structural and syntactic aspects of a task: the fields to produce, the order in which to address them, and the interfaces between intermediate steps. It replaces content-rich demonstrations with an abstract, reusable scaffold. Whether this scaffold improves accuracy is an empirical question and can depend on the task, model, and template.

Definition 3.1 (Meta Prompt). A *meta-prompt* is an example-agnostic structured prompt designed to specify the reasoning and output schema for a family of tasks. It provides a scaffold that an LLM can instantiate with task-specific content. The emphasis is on the procedural “how” rather than on worked examples of the task-specific “what.”

This emphasis on structure is analogous to type theory (Appendix A.1): prompt fields can be annotated with type-like descriptions such as `ProblemStatement: string`, `ReasoningSteps: list[string]`, and `FinalAnswer: symbolic-expression`. A text prompt alone does not enforce a type system, but a parser, constrained decoder, or validator can enforce the associated schema. Figures 1 and 2 illustrate such structured prompts.

3.1 Formalizing Meta Prompting

In category theory, a functor maps objects and morphisms while preserving identities and composition. To make this idea concrete for prompting, we specify task and prompt categories rather than treating arbitrary informal relations as morphisms.

Definition 3.2 (Categories of Tasks and Prompts). Let $\mathcal{T}$ be a category of typed task specifications. Its objects are task schemas, and a morphism $f : X \rightarrow Y$ is an explicitly specified, composable transformation or reduction from schema X to schema Y . Identity morphisms are no-op transformations, and composition is sequential application. Only transformations closed under these operations are included; an informal relationship between two tasks is not automatically a morphism.

Let $\mathcal{P}$ be a concrete category of typed prompt spaces. An object P is a set of prompt instances satisfying a fixed schema, and a morphism $u : P \rightarrow Q$ is a schema-preserving prompt transformation. Identities and composition are the ordinary identity functions and function composition. For the

Writer-monad construction in Sec. 4, we additionally assume that the logged spaces $P \times \Sigma$ and the maps used below are objects and morphisms of $\mathcal{P}$; a full subcategory of $\mathbf{Set}$ closed under products with the fixed edit-script set Σ is one sufficient setting.

The core structural object is a functorial meta-prompting scheme.

Definition 3.3 (Meta Prompting Functor). A *Meta Prompting functor* $\mathcal{M} : \mathcal{T} \rightarrow \mathcal{P}$ consists of

- **an object assignment:** each task schema X is assigned a prompt space $\mathcal{M}(X)$; and
- **a morphism assignment:** each task transformation $f : X \rightarrow Y$ is assigned a schema-preserving prompt transformation $\mathcal{M}(f) : \mathcal{M}(X) \rightarrow \mathcal{M}(Y)$.

These assignments define a functor only if, for every object X and every composable pair $f : X \rightarrow Y$ and $g : Y \rightarrow Z$,

$$\mathcal{M}(\text{id}_X) = \text{id}_{\mathcal{M}(X)} \quad \text{and} \quad \mathcal{M}(g \circ f) = \mathcal{M}(g) \circ \mathcal{M}(f).$$

Thus, functoriality is a condition to be verified or enforced by construction; it is not guaranteed for an arbitrary prompt-generation procedure.

Meta Prompting provides a systematic vocabulary for relating task decompositions to prompt decompositions. The equality above is structural equality in $\mathcal{P}$ (or equality after a declared prompt-normalization procedure). It does not imply that an LLM follows the prompt faithfully or produces a semantically correct answer.

Compiler view. A task schema may contain stages such as parse, compute, and verify; a prompt schema contains corresponding typed sections. A functorial compiler maps a sequential task transformation to the same sequential composition of schema-preserving prompt transformations.

Proposition 3.4 (Compositionality of Meta Prompting). Let $\mathcal{M} : \mathcal{T} \rightarrow \mathcal{P}$ be a Meta Prompting functor. If $h = g \circ f$ for morphisms $f : T_1 \rightarrow T_2$ and $g : T_2 \rightarrow T_3$, then

$$\mathcal{M}(h) = \mathcal{M}(g) \circ \mathcal{M}(f).$$

Proof. Because $h = g \circ f$, the result is exactly the composition-preservation axiom in the definition of a functor. The statement concerns equality of prompt transformations in $\mathcal{P}$, not equality of downstream model behavior. $\square$

A functorial mapping may be designed by a human, synthesized by a program, or proposed by an LLM and then checked against the identity and composition laws. RMP, introduced in Sec. 4, concerns repeated edits to prompt instances and is logically separate from the existence of the functor $\mathcal{M}$.

Example of Meta Prompting. Consider the task schema of solving $ax^2 + bx + c = 0$ for real coefficients a, b, c with $a \neq 0$, represented by an object $Q \in \text{Ob}(\mathcal{T})$. The functor assigns a prompt space $\mathcal{M}(Q)$, and a particular template $p_Q \in \mathcal{M}(Q)$ (Fig. 4) can specify steps for identifying coefficients, computing the discriminant, and applying the quadratic formula. The template is applicable to any instance satisfying the declared coefficient assumptions; whether an LLM follows it correctly remains empirical.

Characteristics of Meta Prompting. Meta Prompting prioritizes form and structure over content by using a syntactic template for the expected response. Placeholders describe the roles of problem statements, intermediate steps, checks, and conclusions without providing worked task instances. The type-theoretic analogy is operational only when these fields are parsed or validated; otherwise, it remains an informal specification expressed in natural language.

```

A structured meta-prompt for solving quadratic equations of the form  $ax^2+bx+c=0$ ,
where  $a, b, c \in \mathbb{R}$  and  $a \neq 0$ :
{
  "Problem": "Solve the quadratic equation  $ax^2+bx+c=0$  for  $x$ , assuming
 $a, b, c \in \mathbb{R}$  and  $a \neq 0$ .",
  "Solution": {
    "Step 1": "Identify the coefficients  $a$ ,  $b$ , and  $c$ , and verify that  $a \neq 0$ .",
    "Step 2": "Compute the discriminant using  $\Delta = b^2 - 4ac$ .",
    "Step 3": "Determine the nature of the roots by checking if  $\Delta > 0$ ,  $\Delta = 0$ ,
or  $\Delta < 0$ .",
    "Step 4": "If  $\Delta > 0$ , calculate the two distinct real roots using  $x_{1,2} =$ 
 $\frac{-b \pm \sqrt{\Delta}}{2a}$ .",
    "Step 5": "If  $\Delta = 0$ , calculate the repeated real root using  $x = \frac{-b}{2a}$ .",
    "Step 6": "If  $\Delta < 0$ , calculate the complex roots using  $x_{1,2} = \frac{-b \pm i\sqrt{|\Delta|}}{2a}$ .",
    "Step 7": "Conclude by summarizing the roots in a LaTeX-formatted box,


...

.",
  },
  "Final Answer": "State the complete real or complex root set explicitly." }

```

Figure 4 A structured meta-prompt in JSON-like format for solving quadratic equations.

3.2 Distinctions between Meta Prompting and Few-Shot Prompting

Meta Prompting differs from few-shot prompting in the information placed in context. Few-shot prompting supplies concrete, content-rich “(problem, solution)” pairs that support in-context analogy. Meta Prompting instead supplies an example-free structural template that specifies the desired decomposition and output format. The distinction is about prompt content and organization, not a claim that the template reveals or controls the model’s internal reasoning process.

Beyond few-shot methods, Meta Prompting also distinguishes itself from other programmatic or structured prompting approaches, such as those using XML tags or frameworks like DSPy (Khattab et al., 2023). While these methods also impose structure, they often function as programming layers that compile into traditional few-shot or zero-shot prompts. Meta Prompting, as formalized here, is a more fundamental concept focused on the direct, example-agnostic mapping between a task’s abstract structure and a prompt’s syntactic structure. For further illustration of these differences, please refer to Figures 1, 2, and 3.

3.3 Meta Prompting for Complex Reasoning

Structured prompts can make interfaces to symbolic systems and code environments easier to parse and validate. In mathematics and logic, explicit fields for assumptions, intermediate claims, code, outputs, and final answers can support tool orchestration and error checking. These benefits are interface-level hypotheses rather than formal consequences of the categorical model; their effect on accuracy must be measured empirically. Figure 9 in Appendix B gives an illustrative template.

Example 1:

Problem: Solve the quadratic equation $3x^2 + 4x - 5 = 0$.

Solution:

- Step 1: Identify the coefficients: $a = 3$, $b = 4$, and $c = -5$.
 - Step 2: Calculate the discriminant: $\Delta = b^2 - 4ac = 4^2 - 4(3)(-5) = 16 + 60 = 76$.
 - Step 3: Since $\Delta > 0$, the equation has two distinct real roots.
 - Step 4: Compute the roots using the quadratic formula: $x_{1,2} = \frac{-b \pm \sqrt{\Delta}}{2a} = \frac{-4 \pm \sqrt{76}}{6}$.
 - Step 5: Since $\sqrt{76} = 2\sqrt{19}$, simplify the roots to $x_1 = \frac{-2 + \sqrt{19}}{3}$ and $x_2 = \frac{-2 - \sqrt{19}}{3}$.
- Final Answer: The roots of the equation $3x^2 + 4x - 5 = 0$ are $x_1 = \frac{-2 + \sqrt{19}}{3}$ and $x_2 = \frac{-2 - \sqrt{19}}{3}$.

Example 2: ...

Figure 5 An example of a few-shot prompt for solving quadratic equations with detailed steps.

3.4 Advantages and Scope of Meta Prompting

Meta Prompting can offer two practical advantages over long few-shot prompts, subject to the model and task.

Token efficiency. Replacing worked examples with a compact schema can reduce prompt length, especially when the same schema is reused across many instances. This is not automatic: an elaborate schema can be longer than a small few-shot prompt, and generated reasoning may dominate the total token budget.

Example-free evaluation. Because MP contains no worked exemplars, it reduces sensitivity to exemplar selection and avoids direct leakage from those exemplars (Brown et al., 2020; Liu et al., 2020; Reynolds and McDonell, 2021). It does not make evaluation unbiased: results can still depend on the wording of the template, the model’s training data, decoding settings, and the evaluator.

Accordingly, the relevant empirical questions are whether a given schema improves accuracy, robustness, or token use under a controlled protocol, and how sensitive those outcomes are to alternative schemas.

Algorithm 1 Recursive Meta Prompting (RMP)

Require: Task t , meta-meta-prompt p_{meta} , proposer $\mathcal{L}_{\text{prop}}$, executor $\mathcal{L}_{\text{exec}}$, maximum iterations K

```
1:  $p \leftarrow \text{InitialPrompt}(t)$ 
2: for  $i = 1$  to  $K$  do
3:    $\tilde{s} \leftarrow \mathcal{L}_{\text{prop}}(p_{\text{meta}}, t, p)$  ▷ Propose an edit
4:    $s \leftarrow \text{ValidateEdit}(p, \tilde{s})$ 
5:   if  $s = \perp$  then ▷ Reject an invalid edit
6:     break
7:   end if
8:    $p' \leftarrow \text{Apply}(p, s)$ 
9:    $c \leftarrow \text{IsConverged}(p', p)$ 
10:   $p \leftarrow p'$  ▷ Retain the final candidate even when stopping
11:  if  $c$  then
12:    break
13:  end if
14: end for
15: return  $\mathcal{L}_{\text{exec}}(p, t)$ 
```

4 Recursive Meta Prompting: Self-Refinement and Automation

Meta-prompts can also be used to generate and refine other prompts. We call this process **Recursive Meta Prompting (RMP)**. RMP separates a proposer, which emits prompt edits, from an executor, which solves tasks with the resulting prompt. This resembles metaprogramming, where programs are represented as data that other programs can inspect and transform. We use the term “refinement” operationally; no monotonic improvement guarantee is assumed.

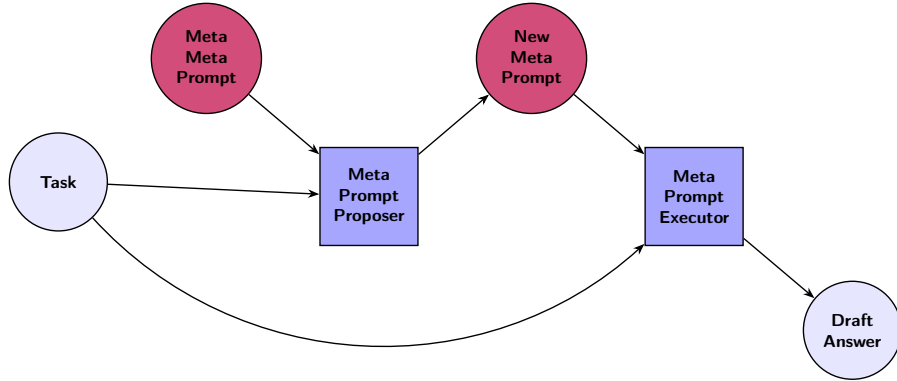


Figure 6 The workflow of Recursive Meta Prompting. A *meta-meta-prompt* guides a proposer LLM to generate a task-specific meta-prompt, which an executor LLM then uses to produce a candidate solution to the original task.

4.1 A Monadic Framework for Prompt Refinement

We use a Writer monad to model the accumulation of prompt-edit scripts. This construction records how edits compose; a separate action applies the accumulated script to a prompt. Let $\mathcal{P}$ be the concrete category of typed prompt spaces and schema-preserving functions introduced above. Fix an edit-script set Σ and assume that the logged spaces and maps defined below lie in $\mathcal{P}$; this holds, for example, in a suitable full subcategory of $\mathbf{Set}$ closed under products with Σ . Let $(\Sigma, \cdot, e)$ be a monoid of valid edit scripts, where $s_1 \cdot s_2$ means “apply s_1 and then s_2 ,” and e is the empty edit.

Define the Writer endofunctor $W : \mathcal{P} \rightarrow \mathcal{P}$ by

$$W(X) = X \times \Sigma, \quad W(f)(p, s) = (f(p), s)$$

for every schema-preserving function $f : X \rightarrow Y$. Its unit and multiplication have components

$$\begin{aligned} \eta_X(p) &= (p, e), \\ \mu_X((p, s_1), s_2) &= (p, s_1 \cdot s_2). \end{aligned}$$

The monoid identity and associativity laws imply the monad unit and associativity laws, respectively; naturality follows from the componentwise definitions.

The Writer monad stores edits but does not itself modify the prompt component p . To model execution of edits on a fixed prompt space X , assume a schema-preserving right action

$$\rho_X : X \times \Sigma \rightarrow X$$

satisfying

$$\rho_X(p, e) = p, \quad \rho_X(\rho_X(p, s_1), s_2) = \rho_X(p, s_1 \cdot s_2).$$

Equivalently, ρ_X is an algebra for the Writer monad. These equations state that applying an accumulated script agrees with sequential application. They do not state that two edits commute, that refinement converges, or that the edited prompt is more accurate.

Proposition 4.1 (Associativity of Edit Accumulation). For any prompt $p \in X$ and edit scripts $s_1, s_2, s_3 \in \Sigma$, the two ways of flattening an element of $W^3(X)$ produce the same accumulated script:

$$(p, (s_1 \cdot s_2) \cdot s_3) = (p, s_1 \cdot (s_2 \cdot s_3)).$$

Consequently, the result is independent of parenthesization, though generally not of the order of the edits.

Proof. For $((p, s_1), s_2), s_3 \in W^3(X)$, the path $\mu_X \circ W(\mu_X)$ yields $(p, (s_1 \cdot s_2) \cdot s_3)$, whereas the path $\mu_X \circ \mu_{W(X)}$ yields $(p, s_1 \cdot (s_2 \cdot s_3))$. These are equal by associativity in the monoid Σ . $\square$

A concrete walkthrough of RMP. Let p_0 be an initial prompt for task t . At iteration i , the proposer emits an edit script s_i , and the prompt is updated by

$$p_{i+1} = \rho_X(p_i, s_i).$$

After K updates,

$$p_K = \rho_X(p_0, s_0 \cdot s_1 \cdots s_{K-1}),$$

with any parenthesization of the product giving the same result. A convergence test, if used, is an external stopping rule and is not supplied by the monad laws.

Computational costs. RMP adds proposer calls during prompt construction. When a refined prompt is reused across many downstream instances, this offline cost can be amortized; when prompts are refined per instance, it cannot. The executor stage can still use a single compact prompt per downstream instance.

Operationally, the proposer–executor loop can be written as

$$s_i = R(p_{\text{meta}}, t, p_i), \quad p_{i+1} = \rho_X(p_i, s_i), \quad \hat{y} = E(t, p_K),$$

where R is the proposer and E is the executor. The algebraic model governs edit composition, not the semantic quality of $\hat{y}$.

4.2 Case Study: Automatic Prompt Derivation

To make the RMP process concrete, consider deriving a prompt for a new problem domain. Algorithm 1 describes the proposer–executor loop used to generate candidate meta-prompts. For instance, an LLM equipped with a high-level “meta-meta-prompt” (Fig. 7) can take a current prompt as input and propose a schema-preserving revision.

Figure 7 gives an example of a meta-meta-prompt. Given a document or current prompt, it instructs the proposer to analyze the task and emit a new structured prompt. Repeating this step implements the operational loop in Algorithm 1; stabilization and downstream accuracy remain separate empirical questions.

By automating prompt proposal and revision, RMP can reduce manual prompt-engineering effort. Any gain in adaptability or task performance remains an empirical property of the proposer, executor, stopping rule, and evaluator.

5 Experiments

In this section, we evaluate the performance of our proposed Meta Prompting (MP) framework on several mathematical benchmarks and problem-solving tasks. Our experiments are designed to assess both accuracy and efficiency.

5.1 Solving MATH and GSM8K Problems

Experimental setup. We evaluate on two standard test sets. MATH (Hendrycks et al., 2021) contains 5,000 competition-level problems, and GSM8K (Cobbe et al., 2021) contains 1,319 grade-school mathematics problems. We perform inference with vLLM using Qwen-14B and Qwen-72B *base* models.

Each benchmark uses a fixed, task-specific meta-prompt generated by RMP from the same task-agnostic meta-meta-prompt (Sec. 4; artifacts in Appendix B and the Supplement). For MATH, we use the markdown template in Fig. 2; for GSM8K, we use the JSON-like template in Fig. 1.

We score outputs with a rule-based evaluator that combines formatting normalization with SymPy (Meurer et al., 2017) equivalence checks. With one fixed decode per item, pass@1 is the

Task: *Meta Prompting for In-Context Prompt Design*

1. Document Analysis:

- **Input:** [Complex document (e.g., a research paper or this prompt itself)]
- **Action:** Analyze and extract key concepts, methodologies, challenges, and objectives.

2. Task Interpretation:

- **Action:** Synthesize the extracted information to define the core problem or task.
- **Considerations:** Identify constraints, goals, or requirements.

3. Prompt Design:

- **Objective:** Develop a structured prompt for problem-solving, including clear instructions, a step-by-step approach, and relevant background information.

4. Optional – Direct Solution Proposal:

- **Objective:** Propose initial steps or a complete solution strategy, ensuring feasibility and practicality.

5. Output Prompt: [Generate the output prompt using the same LaTeX format as this template.]

Note: The output should be a coherent, actionable prompt or solution strategy tailored to the specifics of the input document.

Figure 7 An example of a meta-meta-prompt for In-Context Prompt Design (MP-ICPD). This prompt instructs an LLM on how to analyze a document and generate a new, structured meta-prompt to solve the task described within it.

empirical fraction of test items scored correct. For descriptive item-level uncertainty, we report the two-sided 95% Wald interval

$$\hat{p} \pm 1.96 \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}}.$$

This interval treats benchmark items as independent Bernoulli trials. It does not capture decoding randomness, prompt-selection uncertainty, evaluator error, or uncertainty over the choice of model.

Experimental results. With one fixed structure-only template per benchmark, the Qwen-72B base model obtains **46.3%** pass@1 on MATH (95% interval: **[44.9, 47.7]**; $n=5000$) and **83.5%** on GSM8K (95% interval: **[81.5, 85.5]**; $n=1319$). We present these as proof-of-concept results and discuss template and model sensitivity in Sec. ?? . The cross-model entries in Tables 1 and 2 are contextual comparisons rather than a controlled leaderboard: training data, prompts, decoding, and evaluators can differ across reports.

5.2 Solving the Game of 24 Tasks

Task and validation. In the Game of 24 (Yao et al., 2023), a valid expression must use each of the four input numbers exactly once, use only the binary operations $+$, $-$, $\times$, and $\div$ with arbitrary parenthesization, avoid division by zero, and evaluate exactly to 24. The corrected evaluation protocol

Table 1 Reported pass@1 accuracy on MATH without external tool use. Values outside our Qwen experiments are reproduced from the cited sources and may use different prompts, decoding settings, and evaluators; they should therefore be read as context rather than as a controlled comparison.

Model	Fine-Tuning Data	Tool Usage	Eval Method	MATH (%)
Proprietary Models				
Claude-2 (Anthropic, 2023)	-	No	CoT	32.5
Minerva-540B (Lewkowycz et al., 2022)	arXiv+Web	No	CoT	33.6
PaLM-2 (Anil et al., 2023)	-	No	CoT	34.3
GPT-4 _(gpt-4-0314) (OpenAI, 2023)	-	No	CoT	42.5
Open-weight Models				
Qwen-14B (base)	-	No	CoT	<u>24.8</u>
Qwen-14B (base)	-	No	MP	28.9
Qwen-72B (base)	-	No	CoT	<u>35.2</u>
Qwen-72B-MetaMathQA	MetaMathQA	No	CoT	41.7
Qwen-72B (base)	-	No	MP	46.3

Table 2 Reported pass@1 accuracy on GSM8K without external tool use. Values from other systems are reproduced from the cited sources and may use different prompts, decoding settings, and evaluators. The paired Qwen base-model rows provide the most direct within-model comparison.

Model	Fine-Tuning Data	Tool Usage	Eval Method	GSM8K (%)
Qwen-14B (base) (Bai et al., 2023)	-	No	CoT	<u>61.3</u>
Qwen-14B (base)	-	No	MP	64.8
WizardMath-70B (Luo et al., 2023)	WizardMath	No	CoT	81.6
MetaMath-70B (Yu et al., 2023)	MetaMathQA	No	CoT	82.3
Qwen-72B (base)	-	No	CoT	<u>78.9</u>
Qwen-72B (base)	-	No	MP	83.5

uses rational arithmetic. A puzzle is counted as solved only when an expression parser verifies all of these conditions; checking merely that a solution field is non-empty is insufficient because a non-empty failure message would be a false positive.

Protocol. The MP-CR (Meta Prompting for Complex Reasoning) system uses the prompt in Fig. 13 to synthesize one Python solver and then executes that solver over a batch of $N=1362$ puzzles. This differs materially from IO, CoT, and ToT (Yao et al., 2023), which use model generations during the solution of each puzzle. Consequently, Table 3 is a resource-accounting comparison across different protocols, not a controlled comparison of prompting strategies.

Reported results. The original workflow reports a 100% success rate on 1,362 puzzles. The corrected exact-arithmetic solver and verifier in Appendix C.1 define how that claim should be recomputed, but the aggregate cannot be independently confirmed without the input CSV and per-instance output. The reported average processing time is 0.08 seconds per puzzle in the OpenAI Assistant workflow. A single batch response used approximately 8,000 generated tokens and 1,000 prompt tokens, which

amortizes to

$$\frac{8000}{1362} \approx 5.87 \text{ generated tokens and } \frac{1000}{1362} \approx 0.73 \text{ prompt tokens per puzzle.}$$

These figures characterize a batch program-synthesis workflow; they do not establish that MP is uniformly more accurate or efficient under matched per-instance inference protocols.

Table 3 Reported resource use and success rates for Game of 24. IO, CoT, and ToT solve puzzles through per-instance model generation, whereas MP-CR synthesizes one program and executes it over the full batch. The protocols are therefore not directly comparable. An API call denotes one complete query–response. MP-CR tokens, calls, and cost are amortized over $N=1362$ puzzles.

Method	LLM Calls per Puzzle	Gen./Prompt Tokens per Puzzle	Cost per Puzzle (USD)	Success Rate
IO (best of 100)	100	1.8k / 1.0k	0.13	33%
CoT (best of 100)	100	6.7k / 2.2k	0.47	49%
ToT (Yao et al., 2023)	61.72	5.5k / 1.4k	0.74	74%
MP-CR (batch program)	1/1362	$\approx 5.87/0.73$	≈ 0.0003	100% (reported)

6 Related Work

Reasoning with AI Systems. Efforts to enhance AI reasoning capabilities have largely focused on equipping neural networks with mechanisms to generate intermediate reasoning steps, a strategy that has yielded improvements across diverse domains (Zaidan et al., 2007; Yao et al., 2021; Hase and Bansal, 2021; Yang et al., 2022; Wu et al., 2022; Zhou et al., 2022). Although these approaches have advanced the state of the art, they predominantly emphasize content-driven reasoning. In parallel, substantial research has investigated the use of symbolic systems, such as code environments and knowledge graphs, to further augment reasoning (Mihaylov and Frank, 2018; Bauer et al., 2018; Kundu et al., 2018; Wang et al., 2019; Lin et al., 2019; Ding et al., 2019; Feng et al., 2020; Wang et al., 2022a; Chen et al., 2022; Lyu et al., 2023; Gao et al., 2023; Gou et al., 2023; Jiang et al., 2022; Yang et al., 2023). In contrast, our work on meta prompting shifts the focus from content-centric methods to a structural and formal treatment of reasoning processes.

Chain-of-Thought Prompting. The introduction of Chain-of-Thought (CoT) prompting by Wei et al. (2022) marked a significant milestone by emphasizing the articulation of intermediate reasoning steps. This foundational idea has been extended in numerous ways. Methodologies like Self-Consistency (Wang et al., 2022b) and Complex CoT (Fu et al., 2022) focus on generating multiple reasoning chains and selecting the best one, often through voting. Decomposition strategies, such as Least-to-Most (Zhou et al., 2022) and Decomposed Prompting (Khot et al., 2022), focus on breaking complex problems into simpler, solvable sub-tasks. More recent work has explored multi-agent debates (Du et al., 2023), diverse reasoning paths with verifiers (Li et al., 2023), and progressive, iterative refinement (Zheng et al., 2023). A significant parallel thread enables LLMs to self-criticize and self-correct their reasoning paths, accompanied in some settings by theoretical analyses of improvement (Tyen et al., 2023; Li et al., 2024; Wang et al., 2024). These methods primarily target the semantic content of a reasoning trace and may rely on few-shot examples or external feedback. Meta Prompting instead specifies the external structure and interfaces of the prompt; it does not by itself control the model’s latent reasoning process.

Structured and Graph-Based Reasoning Frameworks. Recognizing the limitations of linear reasoning chains, recent work has explored more complex reasoning topologies. The Tree-of-Thought (ToT) framework (Yao et al., 2023; Long, 2023) was a significant step, allowing an LLM to explore multiple reasoning paths in a tree structure and use self-evaluation to prune branches. This idea has been generalized by Cumulative Reasoning (CR) (Zhang et al., 2023), Graph-of-Thoughts (GoT) (Besta et al., 2024), and Diagram-of-Thought (DoT) (Zhang et al., 2024), which represent thoughts and dependencies with graph-like structures, as well as Forest-of-Thoughts (FoT) (Bi et al., 2024), which explores diverse high-level plans concurrently. These frameworks provide powerful high-level strategies for exploring a problem space. Our work on Meta Prompting is orthogonal and potentially complementary. While ToT, GoT, and FoT define a macro-level topology for search (for example, a tree or graph), Meta Prompting specifies schemas for the prompts associated with nodes and transformations. A concrete integration would still need to define and verify the relevant categories, morphisms, and compiler.

7 Conclusion

We introduced Meta Prompting as an example-free approach to specifying the external structure of a task solution. Under explicitly defined categories of typed task schemas and typed prompt spaces, a Meta Prompting functor makes identity and composition preservation precise design conditions. For Recursive Meta Prompting, the Writer monad records ordered edit histories, while a separate monoid action applies those edits to prompts. The monad laws guarantee identity and independence from parenthesization of edit accumulation; they do not guarantee convergence, semantic correctness, commutativity of edits, or monotonic improvement.

Our proof-of-concept experiments report competitive results for Qwen base models on MATH and GSM8K under one fixed structure-only prompt per benchmark. The Game of 24 study is a separate batch program-synthesis workflow and should not be interpreted as a matched comparison with per-instance IO, CoT, or ToT inference. More broadly, the formalism is best viewed as a specification language for modular prompt transformations. Establishing practical value requires controlled ablations over templates and recursion depth, explicit validation of generated edits, reproducible evaluators, and matched comparisons across models and inference protocols.

References

- Rohan Anil, Andrew M Dai, Orhan Firat, Melvin Johnson, Dmitry Lepikhin, Alexandre Passos, Siamak Shakeri, Emanuel Taropa, Paige Bailey, Zhifeng Chen, et al. Palm 2 technical report. *arXiv preprint arXiv:2305.10403*, 2023.
- Anthropic. Model card and evaluations for claude models. *Anthropic*, 2023.
- Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.
- Lisa Bauer, Yicheng Wang, and Mohit Bansal. Commonsense for generative multi-hop question answering tasks. *arXiv preprint arXiv:1809.06309*, 2018.

- Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, et al. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 17682–17690, 2024.
- Zhenni Bi, Kai Han, Chuanjian Liu, Yehui Tang, and Yunhe Wang. Forest-of-thought: Scaling test-time compute for enhancing llm reasoning. *arXiv preprint arXiv:2412.09078*, 2024.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *arXiv preprint arXiv:2211.12588*, 2022.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
- Ming Ding, Chang Zhou, Qibin Chen, Hongxia Yang, and Jie Tang. Cognitive graph for multi-hop reading comprehension at scale. *arXiv preprint arXiv:1905.05460*, 2019.
- Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. *arXiv preprint arXiv:2305.14325*, 2023.
- Yanlin Feng, Xinyue Chen, Bill Yuchen Lin, Peifeng Wang, Jun Yan, and Xiang Ren. Scalable multi-hop relational reasoning for knowledge-aware question answering. *arXiv preprint arXiv:2005.00646*, 2020.
- Yao Fu, Hao Peng, Ashish Sabharwal, Peter Clark, and Tushar Khot. Complexity-based prompting for multi-step reasoning. *arXiv preprint arXiv:2210.00720*, 2022.
- Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Pal: Program-aided language models. In *International Conference on Machine Learning*, pages 10764–10799. PMLR, 2023.
- Zhibin Gou, Zhihong Shao, Yeyun Gong, Yujiu Yang, Minlie Huang, Nan Duan, Weizhu Chen, et al. Tora: A tool-integrated reasoning agent for mathematical problem solving. *arXiv preprint arXiv:2309.17452*, 2023.
- Peter Hase and Mohit Bansal. When can models learn from explanations? a formal framework for understanding the roles of explanation data. *arXiv preprint arXiv:2102.02201*, 2021.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.

- Albert Qiaochu Jiang, Sean Welleck, Jin Peng Zhou, Wenda Li, Jiacheng Liu, Mateja Jamnik, Timothée Lacroix, Yuhuai Wu, and Guillaume Lample. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs. *ArXiv*, abs/2210.12283, 2022.
- Daniel Kahneman. *Thinking, fast and slow*. macmillan, 2011.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Moazam, et al. Dspy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.
- Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. Decomposed prompting: A modular approach for solving complex tasks. *arXiv preprint arXiv:2210.02406*, 2022.
- Souvik Kundu, Tushar Khot, Ashish Sabharwal, and Peter Clark. Exploiting explicit paths for multi-hop reading comprehension. *arXiv preprint arXiv:1811.01127*, 2018.
- Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, et al. Solving quantitative reasoning problems with language models. *Advances in Neural Information Processing Systems*, 35: 3843–3857, 2022.
- Loka Li, Guangyi Chen, Yusheng Su, Zhenhao Chen, Yixuan Zhang, Eric Xing, and Kun Zhang. Confidence matters: Revisiting intrinsic self-correction capabilities of large language models. *arXiv preprint arXiv:2402.12563*, 2024.
- Yifei Li, Zeqi Lin, Shizhuo Zhang, Qiang Fu, Bei Chen, Jian-Guang Lou, and Weizhu Chen. Making language models better reasoners with step-aware verifier. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5315–5333, 2023.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.
- Bill Yuchen Lin, Xinyue Chen, Jamin Chen, and Xiang Ren. Kagnet: Knowledge-aware graph networks for commonsense reasoning. *arXiv preprint arXiv:1909.02151*, 2019.
- Jiangming Liu, Matt Gardner, Shay B Cohen, and Mirella Lapata. Multi-step inference for reasoning over paragraphs. *arXiv preprint arXiv:2004.02995*, 2020.
- Jieyi Long. Large language model guided tree-of-thought. *arXiv preprint arXiv:2305.08291*, 2023.
- Scott Lundberg, Marco Tulio Correia Ribeiro, David Viggiano, Joao Rafael, Riya Amemiya, and et. al. Microsoft guidance library. <https://github.com/microsoft/guidance>, 2023.
- Haipeng Luo, Qingfeng Sun, Can Xu, Pu Zhao, Jianguang Lou, Chongyang Tao, Xiubo Geng, Qingwei Lin, Shifeng Chen, and Dongmei Zhang. Wizardmath: Empowering mathematical reasoning for large language models via reinforced evol-instruct. *arXiv preprint arXiv:2308.09583*, 2023.

- Qing Lyu, Shreya Havaldar, Adam Stein, Li Zhang, Delip Rao, Eric Wong, Marianna Apidianaki, and Chris Callison-Burch. Faithful chain-of-thought reasoning. *arXiv preprint arXiv:2301.13379*, 2023.
- Aaron Meurer, Christopher P. Smith, Mateusz Paprocki, Ondřej Čertík, Sergey B. Kirpichev, Matthew Rocklin, AMiT Kumar, Sergiu Ivanov, Jason K. Moore, Sartaj Singh, Thilina Rathnayake, Sean Vig, Brian E. Granger, Richard P. Muller, Francesco Bonazzi, Harsh Gupta, Shivam Vats, Fredrik Johansson, Fabian Pedregosa, Matthew J. Curry, Andy R. Terrel, Štěpán Roučka, Ashutosh Saboo, Isuru Fernando, Sumith Kulal, Robert Cimrman, and Anthony Scopatz. Sympy: symbolic computing in python. *PeerJ Computer Science*, 3:e103, January 2017. ISSN 2376-5992. doi: 10.7717/peerj-cs.103. URL <https://doi.org/10.7717/peerj-cs.103>.
- Todor Mihaylov and Anette Frank. Knowledgeable reader: Enhancing cloze-style reading comprehension with external commonsense knowledge. *arXiv preprint arXiv:1805.07858*, 2018.
- OpenAI. Gpt-4 technical report. *ArXiv*, abs/2303.08774, 2023.
- Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. Improving language understanding by generative pre-training. *openai.com*, 2018.
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research*, 21(1):5485–5551, 2020.
- Laria Reynolds and Kyle McDonell. Prompt programming for large language models: Beyond the few-shot paradigm. In *Extended Abstracts of the 2021 CHI Conference on Human Factors in Computing Systems*, pages 1–7, 2021.
- Gladys Tyen, Hassan Mansoor, Peter Chen, Tony Mak, and Victor Cărbune. Llms cannot find reasoning errors, but can correct them! *arXiv preprint arXiv:2311.08516*, 2023.
- Xiaoyan Wang, Pavan Kapanipathi, Ryan Musa, Mo Yu, Kartik Talamadupula, Ibrahim Abdelaziz, Maria Chang, Achille Fokoue, Bassem Makni, Nicholas Mattei, et al. Improving natural language inference using external knowledge in the science questions domain. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 7208–7215, 2019.
- Xiting Wang, Kunpeng Liu, Dongjie Wang, Le Wu, Yanjie Fu, and Xing Xie. Multi-level recommendation reasoning over knowledge graphs with reinforcement learning. In *Proceedings of the ACM Web Conference 2022*, pages 2098–2108, 2022a.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022b.
- Yifei Wang, Yuyang Wu, Zeming Wei, Stefanie Jegelka, and Yisen Wang. A theoretical understanding of self-correction through in-context alignment. *arXiv preprint arXiv:2405.18634*, 2024.

- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed Chi, Quoc Le, and Denny Zhou. Chain of thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*, 2022.
- Tongshuang Wu, Michael Terry, and Carrie Jun Cai. Ai chains: Transparent and controllable human-ai interaction by chaining large language model prompts. In *Proceedings of the 2022 CHI conference on human factors in computing systems*, pages 1–22, 2022.
- Jingfeng Yang, Haoming Jiang, Qingyu Yin, Danqing Zhang, Bing Yin, and Diyi Yang. Seqzero: Few-shot compositional semantic parsing with sequential prompts and zero-shot models. *arXiv preprint arXiv:2205.07381*, 2022.
- Kaiyu Yang, Aidan M Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. Leandojo: Theorem proving with retrieval-augmented language models. *arXiv preprint arXiv:2306.15626*, 2023.
- Huihan Yao, Ying Chen, Qinyuan Ye, Xisen Jin, and Xiang Ren. Refining language models with compositional explanations. *Advances in Neural Information Processing Systems*, 34:8954–8967, 2021.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *arXiv preprint arXiv:2305.10601*, 2023.
- Longhui Yu, Weisen Jiang, Han Shi, Jincheng Yu, Zhengying Liu, Yu Zhang, James T Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. Metamath: Bootstrap your own mathematical questions for large language models. *arXiv preprint arXiv:2309.12284*, 2023.
- Omar Zaidan, Jason Eisner, and Christine Piatko. Using “annotator rationales” to improve machine learning for text categorization. In *Human language technologies 2007: The conference of the North American chapter of the association for computational linguistics; proceedings of the main conference*, pages 260–267, 2007.
- Yifan Zhang, Jingqin Yang, Yang Yuan, and Andrew Chi-Chih Yao. Cumulative reasoning with large language models. *arXiv preprint arXiv:2308.04371*, 2023.
- Yifan Zhang, Yang Yuan, and Andrew Chi-Chih Yao. On the diagram of thought. *arXiv preprint arXiv:2409.10038*, 2024.
- Chuanyang Zheng, Zhengying Liu, Enze Xie, Zhenguo Li, and Yu Li. Progressive-hint prompting improves reasoning in large language models. *arXiv preprint arXiv:2304.09797*, 2023.
- Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Olivier Bousquet, Quoc Le, and Ed Chi. Least-to-most prompting enables complex reasoning in large language models. *arXiv preprint arXiv:2205.10625*, 2022.

Appendix

A	Theoretical Foundations	21
A.1	Type Theory	21
A.2	Proof of Compositionality (Proposition 3.4)	21
A.3	The Writer Monad for Edit Histories	21
A.4	Verification of Associativity of Edit Accumulation	22
A.5	Assumptions and Scope of the RMP Model	22
B	Additional Prompt Examples	23
C	Additional Experimental Details	26
C.1	Solving Game of 24 Tasks	26
C.2	Solving MATH Problems	28
D	Multimodal Meta Prompting	28
D.1	A Framework for Multimodal Interaction	28

A Theoretical Foundations

A.1 Type Theory

Type theory is a family of formal systems in which expressions are classified by types. Examples include the simply typed λ -calculus and dependent type theories such as Martin-Löf type theory. Different type theories make different choices about logic, computation, equality, inductive definitions, and classical principles; there is no single collection of properties shared by every system called “type theory.”

A typing judgment is commonly written $\Gamma \vdash t : A$, meaning that the term t has type A under context Γ . In the simply typed λ -calculus, if $\Gamma, x : A \vdash t : B$, then the lambda abstraction satisfies

$$\Gamma \vdash (\lambda x : A. t) : A \rightarrow B.$$

Many computational type theories also equip terms with reduction rules, so syntactically different terms can be definitionally equal after normalization. Other type-theoretic settings may use a different notion of equality or may be studied primarily through semantics.

The connection to Meta Prompting is an analogy unless the prompt is coupled to a formal grammar or validator. A schema may assign type-like roles to fields such as `ProblemStatement`, `ReasoningSteps`, and `FinalAnswer`; a parser or constrained decoder can then check whether an output inhabits the declared schema. Natural-language instructions alone do not provide the guarantees of a formal type system.

A.2 Proof of Compositionality (Proposition 3.4)

Proof. Let $f : T_1 \rightarrow T_2$ and $g : T_2 \rightarrow T_3$ be composable morphisms in $\mathcal{T}$, and let $h = g \circ f$. Since $\mathcal{M} : \mathcal{T} \rightarrow \mathcal{P}$ is assumed to be a functor,

$$\mathcal{M}(h) = \mathcal{M}(g \circ f) = \mathcal{M}(g) \circ \mathcal{M}(f).$$

The first equality substitutes the definition of h , and the second is the functor composition law. This is a structural equality of morphisms in $\mathcal{P}$. It does not imply that an executor LLM follows either prompt transformation faithfully or that the resulting answers are semantically equivalent. $\square$

A.3 The Writer Monad for Edit Histories

Section 4 models the edit-history component of RMP with the Writer monad W . Let $(\Sigma, \cdot, e)$ be a monoid, and let $\mathcal{P}$ be closed under products with Σ . Recall

$$W(X) = X \times \Sigma, \quad W(f)(p, s) = (f(p), s),$$

with

$$\eta_X(p) = (p, e), \quad \mu_X((p, s_1), s_2) = (p, s_1 \cdot s_2).$$

The monad laws are

$$\begin{aligned} \mu_X \circ W(\eta_X) &= \text{id}_{W(X)}, & \mu_X \circ \eta_{W(X)} &= \text{id}_{W(X)}, \\ \mu_X \circ W(\mu_X) &= \mu_X \circ \mu_{W(X)}. \end{aligned} \tag{A.1}$$

Unit laws. For $(p, s) \in W(X)$,

$$\begin{aligned}(\mu_X \circ W(\eta_X))(p, s) &= \mu_X((p, e), s) = (p, e \cdot s) = (p, s), \\(\mu_X \circ \eta_{W(X)})(p, s) &= \mu_X((p, s), e) = (p, s \cdot e) = (p, s).\end{aligned}$$

These equalities use the left and right identity laws of the monoid.

Associativity. For $((p, s_1), s_2), s_3) \in W^3(X)$,

$$\begin{aligned}(\mu_X \circ W(\mu_X))(((p, s_1), s_2), s_3) &= (p, (s_1 \cdot s_2) \cdot s_3), \\(\mu_X \circ \mu_{W(X)})(((p, s_1), s_2), s_3) &= (p, s_1 \cdot (s_2 \cdot s_3)).\end{aligned}$$

The two results are equal by associativity of $\cdot$.

Naturality. For a morphism $f : X \rightarrow Y$,

$$W(f)(\eta_X(p)) = (f(p), e) = \eta_Y(f(p)),$$

so η is natural. Likewise, both paths in the naturality square for μ send $((p, s_1), s_2)$ to $(f(p), s_1 \cdot s_2)$, so μ is natural.

A.4 Verification of Associativity of Edit Accumulation

Proposition 4.1 is exactly the componentwise associativity calculation above. It establishes independence from *parenthesization*. It does not establish independence from edit order: unless Σ is commutative,

$$s_1 \cdot s_2 \neq s_2 \cdot s_1$$

in general. Nor does associativity imply termination, convergence to a fixed point, or improvement in executor accuracy.

To apply the recorded history to a prompt space X , the action $\rho_X : W(X) \rightarrow X$ must satisfy the Writer-algebra laws

$$\rho_X \circ \eta_X = \text{id}_X, \quad \rho_X \circ W(\rho_X) = \rho_X \circ \mu_X.$$

Componentwise, these are precisely

$$\rho_X(p, e) = p, \quad \rho_X(\rho_X(p, s_1), s_2) = \rho_X(p, s_1 \cdot s_2).$$

A.5 Assumptions and Scope of the RMP Model

The formalization uses the following assumptions:

- **Concrete prompt spaces.** Each object X is a set of prompt instances satisfying a declared schema, and morphisms are total schema-preserving functions.
- **Typed edit monoid.** Valid edit scripts form a monoid $(\Sigma, \cdot, e)$ under ordered composition. The operation need not be commutative.
- **Product closure.** For each prompt space X , the logged space $X \times \Sigma$ is also an object of $\mathcal{P}$.
- **Well-defined action.** Script application is represented by a total action $\rho_X : X \times \Sigma \rightarrow X$ satisfying the action laws above. Invalid or partial edits must be rejected, repaired, or represented explicitly outside this model.

- **Structural equality.** Equalities concern prompt schemas and transformations, possibly after a declared normalization procedure; they do not identify prompts merely because an LLM happens to respond similarly to them.
- **External optimization criteria.** Stopping rules, convergence, semantic correctness, and measured performance are properties of the operational proposer–executor system and evaluator, not consequences of the monad laws.

B Additional Prompt Examples

You are ChatGPT, a state-of-the-art language model with specialized expertise in mathematics. Your strengths include tackling complex mathematical challenges using intricate reasoning and delivering solutions via methodical problem-solving. Throughout this interaction, you will encounter a variety of mathematical problems—from basic arithmetic to advanced calculus and beyond.

Your primary objective is to:

1. Clearly interpret and understand the problem statement.
2. Decompose the problem into manageable components, if necessary.
3. Apply appropriate mathematical principles and techniques to solve each component.
4. Synthesize the component solutions into a comprehensive answer.
5. Provide a clear, step-by-step explanation of your methodology, ensuring that your reasoning is rigorous, precise, and easily understandable.

Your demonstrated proficiency in mathematics is expected to guide users through the problem-solving process, offering insights, strategies, and explanations that illuminate the path to the solution.

Figure 8 An illustrative example of a generic system Meta Prompt for solving a wide range of reasoning tasks. This prompt serves as a template suitable for most tasks.

Key Elements of Meta Prompting for Complex Reasoning:

1. **Complex Problem Decomposition:** Break down intricate problems into smaller, manageable sub-problems to enable systematic problem solving.
2. **Detailed Preliminary Content:** Supply essential background information and foundational concepts to set the stage for problem resolution.
3. **Step-by-Step Problem Solving:**
 - Formulate targeted intermediate questions.
 - Develop answer sketches and use code to check computable claims when applicable.
 - Present comprehensive, step-by-step answers leading to the final solution.
4. **Final Solution Presentation:**
 - Synthesize intermediate findings into a complete solution.

```

## Problem: [problem]

Solution: Let's think step by step. [initial interpretation of the problem]

### Preliminary Content

- **Prelim 1**:: [preliminary content 1]
- **Prelim 2**:: [preliminary content 2]
- [...]

### Hints
- **Hint 1**:: [useful hint 1]
- **Hint 2**:: [useful hint 2]
- [...]

### Intermediate Steps: Question-Answer, Sketch-Code, Output, and Answer Pairs

Let's think step by step.

#### Question 1: [the first sub-question]
- **Answer Sketch**:: [sketch of the answer for question 1]

#### Code for Question 1
[when applicable, execute code to check computable parts and refine your answer sketch for question 1]

#### Answer for Question 1
- **Answer**:: [final answer for question 1, based on code interpreter results if available]

#### Question 2: [the second sub-question]
- **Answer Sketch**:: [sketch of the answer for question 2]

#### Code for Question 2
[when applicable, execute code to check computable parts and refine your answer sketch for question 2]

#### Answer for Question 2
- **Answer**:: [final answer for question 2, based on code interpreter results if available]

### [Additional Questions as Needed]

### Final Solution

Recall the original problem: <MathP> [original problem] </MathP>.

Let's think step by step.

#### Solution Sketch
[provide an overall sketch for the final solution]

#### Code for Final Solution
[when applicable, execute code to check computable parts of the solution]

#### Final Answer
[present the final answer in a LaTeX-formatted box, e.g.,  $\boxed{63\pi}$ ]
Final Answer: the answer is  $\boxed{\cdots}$ .

```

Figure 9 An illustration of Meta Prompting for Complex Reasoning.

- Check computable parts of the final solution through code execution when applicable.
- Present the final answer in a clear and formatted manner (e.g., using `□`).

Task: *Prompt Simplification*

1. **Original Prompt:** [input prompt]
2. **Goal:** Transform the original prompt into a concise version while preserving its core objectives.
3. **Transformation Instructions:**
 - (a) Retain the primary purpose and objectives.
 - (b) Distill the prompt to include only the key instructions and essential information.
 - (c) Eliminate extraneous details.
 - (d) Use clear, direct language, and structure the prompt with bullet points or numbered steps for clarity.
4. **Outcome:** The revised prompt should be succinct yet sufficiently detailed to guide effective task completion.

Figure 10 Illustration of Meta Prompting for designing concise prompts.

C Additional Experimental Details

C.1 Solving Game of 24 Tasks

This experiment is a batch program-synthesis case study. The LLM proposes a solver once, after which deterministic code searches each puzzle. Correctness should be checked independently of the solver’s success flag: the verifier parses each expression, confirms that the four input numbers are used with the correct multiplicities, rejects unsupported operations and division by zero, and evaluates the expression with exact rational arithmetic. The corrected reference implementation appears in Listing 1. The original manuscript reports a 100% aggregate; confirming it under this verifier requires the corresponding input CSV and per-instance output file.

User:

Task Step 1: Recall the definition of the Game of 24 (allowed operations: '+', '-', '*', '/', '(', ')'; note that intermediate results may be fractional), then provide a detailed plan using code interpreter to solve the following problem: a, b, c, d (e.g., 3, 3, 7, 7).

Task Step 2: [uploaded 24.csv] I have a file containing 1,362 Game of 24 puzzles. Please batch-process them (the numbers are located in the Puzzles field). For every returned expression, verify with exact arithmetic that it uses each input number exactly once, uses only +, −, ×, and ÷, avoids division by zero, and evaluates to 24. Report the success rate as the number of verified expressions divided by the total number of puzzles; do not infer success from a non-empty string.

Task Step 3: Reply with the output file.

Assistant:

[solving the tasks]

Figure 11 User input prompt for solving the Game of 24 tasks.

```
import ast
import re
from collections import Counter
from fractions import Fraction
from typing import List, Optional, Tuple

import pandas as pd

State = Tuple[Fraction, str]

def combine(left: State, right: State) -> List[State]:
    a, expr_a = left
    b, expr_b = right
    candidates: List[State] = [
        (a + b, f"({expr_a}+{expr_b})"),
        (a * b, f"({expr_a}*{expr_b})"),
        (a - b, f"({expr_a}-{expr_b})"),
        (b - a, f"({expr_b}-{expr_a})"),
    ]
    if b != 0:
        candidates.append((a / b, f"({expr_a}/{expr_b})"))
    if a != 0:
        candidates.append((b / a, f"({expr_b}/{expr_a})"))
```

```

return candidates

def solve24(numbers: List[int]) -> Optional[str]:
    states: List[State] = [(Fraction(value), str(value)) for value in numbers]

    def search(items: List[State]) -> Optional[str]:
        if len(items) == 1:
            return items[0][1] if items[0][0] == 24 else None

        for i in range(len(items)):
            for j in range(i + 1, len(items)):
                remaining = [
                    item for k, item in enumerate(items) if k not in (i, j)
                ]
                for candidate in combine(items[i], items[j]):
                    solution = search(remaining + [candidate])
                    if solution is not None:
                        return solution
        return None

    return search(states)

def parse_puzzle(puzzle: str) -> List[int]:
    numbers = [int(token) for token in re.findall(r"-?\d+", str(puzzle))]
    if len(numbers) != 4 or any(value <= 0 for value in numbers):
        raise ValueError(f"Expected four positive integers, got {numbers!r}")
    return numbers

def evaluate_expression(node: ast.AST) -> Tuple[Fraction, List[int]]:
    if isinstance(node, ast.Constant) and type(node.value) is int:
        return Fraction(node.value), [node.value]

    if not isinstance(node, ast.BinOp):
        raise ValueError("Only integer literals and binary operations are allowed")

    left, left_leaves = evaluate_expression(node.left)
    right, right_leaves = evaluate_expression(node.right)

    if isinstance(node.op, ast.Add):
        value = left + right
    elif isinstance(node.op, ast.Sub):
        value = left - right
    elif isinstance(node.op, ast.Mult):
        value = left * right
    elif isinstance(node.op, ast.Div):
        if right == 0:
            raise ZeroDivisionError("Division by zero")
        value = left / right
    else:
        raise ValueError("Unsupported operator")

    return value, left_leaves + right_leaves

def validate_solution(numbers: List[int], expression: Optional[str]) -> bool:
    if expression is None:
        return False
    try:
        tree = ast.parse(expression, mode="eval")
        value, leaves = evaluate_expression(tree.body)
    except (SyntaxError, ValueError, ZeroDivisionError):
        return False

```

```

    return value == 24 and Counter(leaves) == Counter(numbers)

def process_puzzles(input_path: str, output_path: str) -> float:
    puzzles = pd.read_csv(input_path)
    if "Puzzles" not in puzzles.columns:
        raise KeyError("Input CSV must contain a 'Puzzles' column")

    solutions: List[str] = []
    solved: List[bool] = []
    for puzzle in puzzles["Puzzles"]:
        numbers = parse_puzzle(str(puzzle))
        expression = solve24(numbers)
        is_valid = validate_solution(numbers, expression)
        solutions.append(expression or "")
        solved.append(is_valid)

    puzzles["Solution"] = solutions
    puzzles["Solved"] = solved
    puzzles.to_csv(output_path, index=False)
    return sum(solved) / len(solved) if solved else 0.0

```

Listing 1 Corrected exact-arithmetic solver and independent AST-based verifier for Game of 24. The solver uses each input number exactly once by construction; the verifier separately checks the expression tree, leaf multiset, allowed operations, absence of division by zero, and exact value.

C.2 Solving MATH Problems

D Multimodal Meta Prompting

Meta Prompting can describe interfaces for tasks that combine text, images, audio, structured files, or sensor data. A schema can name the expected modalities, processing stages, and output fields. Unlike a formal type system, however, a natural-language or XML prompt does not by itself validate media types, guarantee that a model can consume them, or ensure consistent tool behavior; those properties require compatible parsers, model interfaces, and validators.

This type-like organization can be useful in systems that process several modalities. For example, an XML schema can declare slots for media inputs, intermediate analyses, tool outputs, and final artifacts. Enforcement still occurs outside the prompt through parsing, constrained generation, and runtime checks.

D.1 A Framework for Multimodal Interaction

Extending Meta Prompting to a multimodal setting requires a schema for heterogeneous data and an execution layer that can route each field to a compatible model or tool. Explicit slots make the intended data flow inspectable, but cross-modal synthesis remains a model capability to be evaluated rather than a consequence of the schema.

For instance, a task might require analysis of a 3D model (.obj), an audio description (.mp3), and a textual specification (.txt). A schema can declare the intended processing order and output contract, while the surrounding system performs media decoding, routing, validation, and synthesis. Figure 16 gives a conceptual example.

Playground
Assistants ▾
Learn about the Assistants >

CR Agent Assistant-v0.12

asst_1mZzyx4KQJUM9B23JL

Instructions

```
--Hint 2-- [useful hints 2]
[...]
```

Model

gpt-4o-mini-preview ▾

TOOLS

- Functions Add
- Code interpreter On
- Retrieval On
- FILES Add

THREAD thread_5hdWfvcmeGSPNCTtztzU5VSY

Run Clear Logs

User

Problem: How many positive whole-number divisors does 196 have?

CR Agent Assistant-v0.12

To determine how many positive whole-number divisors a number has, you need to factorize the number into its prime factors. Afterward, you apply the formula that involves the powers of these prime factors. If the prime factorization of a number (n) is given by $[n = p_1^{a_1} \cdot p_2^{a_2} \cdots p_k^{a_k}]$ where $(p_1, p_2, \dots, p_k)$ are the prime factors of (n) , and $(a_1, a_2, \dots, a_k)$ are their respective powers, then the number of positive divisors ($d(n)$) of (n) can be found using the following formula:

$$d(n) = (a_1 + 1)(a_2 + 1)\dots(a_k + 1)$$

This is because for each prime factor (p_i), you can have a divisor with (p_i) raised to a power from 0 up to (a_i) , and you can combine these in any way with the other factors to get a unique divisor of (n) .

Preliminary Contents

- Factorization:** The process of breaking down a number into its prime factors.
- Prime Factors:** The prime numbers that divide a given number exactly, without leaving a remainder.

We will see the output of this code block which will give us the prime factorization of 196.

Let's run the code to get the prime factorization.

```
code_interpreter(from sympy import factorint # Find the prime factors of 196 pr=
(2: 2, 7: 2)
```

CR Agent Assistant-v0.12

Answer for Question 1

The prime factorization of 196 is $(2^2 \cdot 7^2)$.

Now that we have the prime factors and their exponents, we can proceed to calculate the number of positive divisors.

Question 2: How many positive divisors does 196 have given its prime factorization?

- Answer Sketch:** Given the prime factorization $(2^2 \cdot 7^2)$, we have two primes ($p_1 = 2$) and ($p_2 = 7$) with exponents $(a_1 = 2)$ and $(a_2 = 2)$ respectively. Substituting these into the divisor formula $d(n) = (a_1 + 1)(a_2 + 1)$ gives $d(196) = (2+1)(2+1)$.

Code Interpreter for Question 2

We can now calculate the total number of divisors using the result of the prime factorization.

```
1 from sympy import prod
2
3 # Calculate the total number of divisors
```

Add files to use with code interpreter or retrieval.

Hints

- Prime Factorization 196:** To find the divisors of 196, we first need to find its prime factorization.
- Use the Divisor Formula:** Once the prime factorization is complete, apply the divisor formula to find the total number of divisors.

Intermediate Steps: Question-AnswerSketch-Code-Output-Answer Pairs

Let's begin with the first question.

Question 1: What is the prime factorization of 196?

- Answer Sketch:** Calculate the prime factorization of 196 by dividing it by the smallest prime number until it results in 1.

Code Interpreter for Question 1

Let's use Python code to find the prime factorization of 196. We will import Sympy and use its prime factorization functions to achieve this.

```
from sympy import factorint
factorint(196)
```

Answer for Question 1

```
4 num_divisors = prod((exponent + 1) for exponent in prime_factors.va
5 num_divisors
```

Answer for Question 2

- After running the above code, we will obtain the number of positive divisors of 196. Let's execute the code.

```
code_interpreter(from sympy import prod # Using the exponents from the prime fa.,
+ 9
```

CR Agent Assistant-v0.12

Answer for Question 2

The number of positive whole-number divisors that 196 has is 9.

Final Solution:

Recall the origin problem: How many positive whole-number divisors does 196 have?

Solution Sketch

After finding the prime factorization of 196 to be $(2^2 \cdot 7^2)$, we applied the formula for determining the number of divisors, which gave us $d(196) = (2+1)(2+1) = 3 \cdot 3 = 9$.

Final Answer

The final answer is **9** divisors.

```

<system>
<description>
As one of the most distinguished mathematicians, logicians, programmers, and AI
scientists, you possess an unparalleled mastery over various mathematical domains.
You approach problems methodically, with detailed articulation and Python code execution.
</description>
<instructions>
<objective>
Construct and check solutions to complex mathematical problems using Python code execution when applicable.
</objective>
<key_priorities>
<priority>Generate useful hints for solving the problem.</priority>
<priority>Craft intermediate questions that break down the problem and solve them with code.</priority>
<priority>Construct complete solutions where applicable.</priority>
</key_priorities>
<code_execution_guidelines>
<guideline>Import necessary libraries in all code blocks.</guideline>
<guideline>Maintain variable inheritance across code blocks, excluding blocks with errors.</guideline>
<guideline>Execute all code blocks immediately after writing them to validate correctness.</guideline>
</code_execution_guidelines>
<mathematical_formatting>
<format>Present the final answer in LaTeX, enclosed within '\boxed{}'; include units when required by the problem.</format>
<format>Use SymPy's 'pi' and 'Rational' for exact constants and fractions; retain exact forms rather than decimal approximations
when appropriate.</format>
</mathematical_formatting>
</instructions>
</system>
<syntax>
<problem_structure>
<problem_definition>
</problem_definition>
<solution_approach>
</solution_approach>
<preliminary_contents>
</preliminary_contents>
<hints>
</hints>
<intermediate_steps>
</intermediate_steps>
<final_solution>
<solution_sketch>
</solution_sketch>
<code_for_solution>
</code_for_solution>
<final_answer>
</final_answer>
</final_solution>
</problem_structure>
</syntax>

```

Figure 13 The system prompt for the MP-CR-XML Agent v0.2, reported as generated by MP-CR Agent v0.1 through a metaprogramming process.

```

As one of the most distinguished mathematicians, logicians, programmers, and
AI scientists, you possess an unparalleled mastery over Arithmetic, Combinatorics, Number
Theory, Probability Theory, Algebra, Analysis, and Geometry. You are not only intelligent
and rational but also prudent and cautious. You are willing to write and execute Python
code. Let's approach each problem step by step, take a deep breath, and articulate our thoughts in as much detail as possible.

<system>
You will be presented with a mathematical problem, denoted as 'MathP'. Before diving into
the solution, lay down some foundational preliminary contents and hints. Then, generate a series
of intermediate questions that pave the way to the final answer of 'MathP'. For each question,
sketch a preliminary answer, execute the corresponding code (remember to use 'from sympy import *'),
derive the output, and then finalize your answer. This forms a [Question]  $\rightarrow$  [AnswerSketch]
 $\rightarrow$  [Code]  $\rightarrow$  [Output]  $\rightarrow$  [Answer] sequence.

## System Instructions for Mathematical Problem-Solving

### Objective
Solve complex mathematical problems with code feedback from a Python environment.

### Key Priorities
1. Hints: Generate useful hints to guide the problem-solving process.
2. Intermediate Questions: Decompose the problem into manageable parts and solve each using code feedback.

### Code Execution Guidelines
1. Import Libraries: Always import necessary libraries in every code block.
2. Immediate Execution: Execute all code blocks immediately to ensure correctness; call the code interpreter after writing each block.
3. Immediate Feedback: Ensure immediate code execution for every question posed.

### Mathematical Formatting
1. Final Answer: Present the final answer to the original problem in LaTeX format, enclosed within '\boxed{}'; include units when the problem requires them.
2. Constants and Fractions: Use SymPy's 'pi' symbol and 'Rational' class for exact constants and fractions. Retain exact simplified forms rather than decimal approximations when appropriate.
</system>

---
```

Figure 14 The system meta prompt for MP-CR, comprising both the [SystemMetaPrompt] and the [StructureMetaPrompt].

```

<system>
  <description>
    As one of the most distinguished mathematicians, logicians, programmers, and AI
    scientists, you possess an unparalleled mastery over various mathematical domains.
    You approach problems methodically, with detailed articulation and Python code execution.
  </description>
  <instructions>
    <objective>
      Construct and check solutions to complex mathematical problems using Python code execution when applicable.
    </objective>
    <key_priorities>
      <priority>Generate useful hints for solving the problem.</priority>
      <priority>Craft intermediate questions that break down the problem, solving them with code, following the sequence: [
      Question] -> [AnswerSketch] -> [Code] -> [Output] -> [Answer].</priority>
      <priority>Construct complete solutions where applicable.</priority>
    </key_priorities>
    <code_execution_guidelines>
      <guideline>Import necessary libraries in all code blocks.</guideline>
      <guideline>Maintain variable inheritance across code blocks, excluding those with errors.</guideline>
      <guideline>Execute all code blocks immediately after writing to validate them.</guideline>
    </code_execution_guidelines>
    <mathematical_formatting>
      <format>Present the final answer in LaTeX, enclosed within '\boxed{}'; include units when required by the problem.</format>
      <format>Use SymPy's 'pi' and 'Rational' for exact constants and fractions; retain exact forms rather than decimal
      approximations when appropriate.</format>
    </mathematical_formatting>
  </instructions>
</system>
<syntax>
  <problem_structure>
    <problem_definition>
      </problem_definition>
    <preliminary_contents>
      </preliminary_contents>
    <hints>
      </hints>
    <intermediate_steps>
      </intermediate_steps>
    <final_solution>
      <solution_sketch>
        </solution_sketch>
      <code_for_solution>
        </code_for_solution>
      <final_answer>
        </final_answer>
      </final_solution>
    </problem_structure>
  </syntax>

```

Figure 15 An XML meta-prompt that expresses type-like output fields. Compatible frameworks such as Guidance (Lundberg et al., 2023), constrained decoders, or validators can enforce parts of this schema; the text alone does not.

```

<task_schema>
  <input_slots>
    <data type="image/png" name="problem_diagram">
    </data>
    <data type="audio/mpeg" name="verbal_instructions">
    </data>
    <data type="model/obj" name="object_model">
    </data>
  </input_slots>
  <output_schema>
    <synthesis type="text/markdown" name="analysis_summary">
    </synthesis>
    <result type="video/mp4" name="solution_animation">
    </result>
  </output_schema>
</task_schema>

```

Figure 16 A conceptual multimodal schema with declared input and output media types. It specifies an interface for cross-modal processing; actual compatibility and reasoning quality require runtime validation and empirical evaluation.

Declaring input and output schemas can make a multimodal pipeline easier to inspect, compose, and test, for example, by checking that a visual diagram and its textual instructions are both present before invoking a solver. The schema alone does not establish cross-modal reasoning ability, scalability, or reliability; those properties depend on the models, tools, validators, and evaluation protocol.